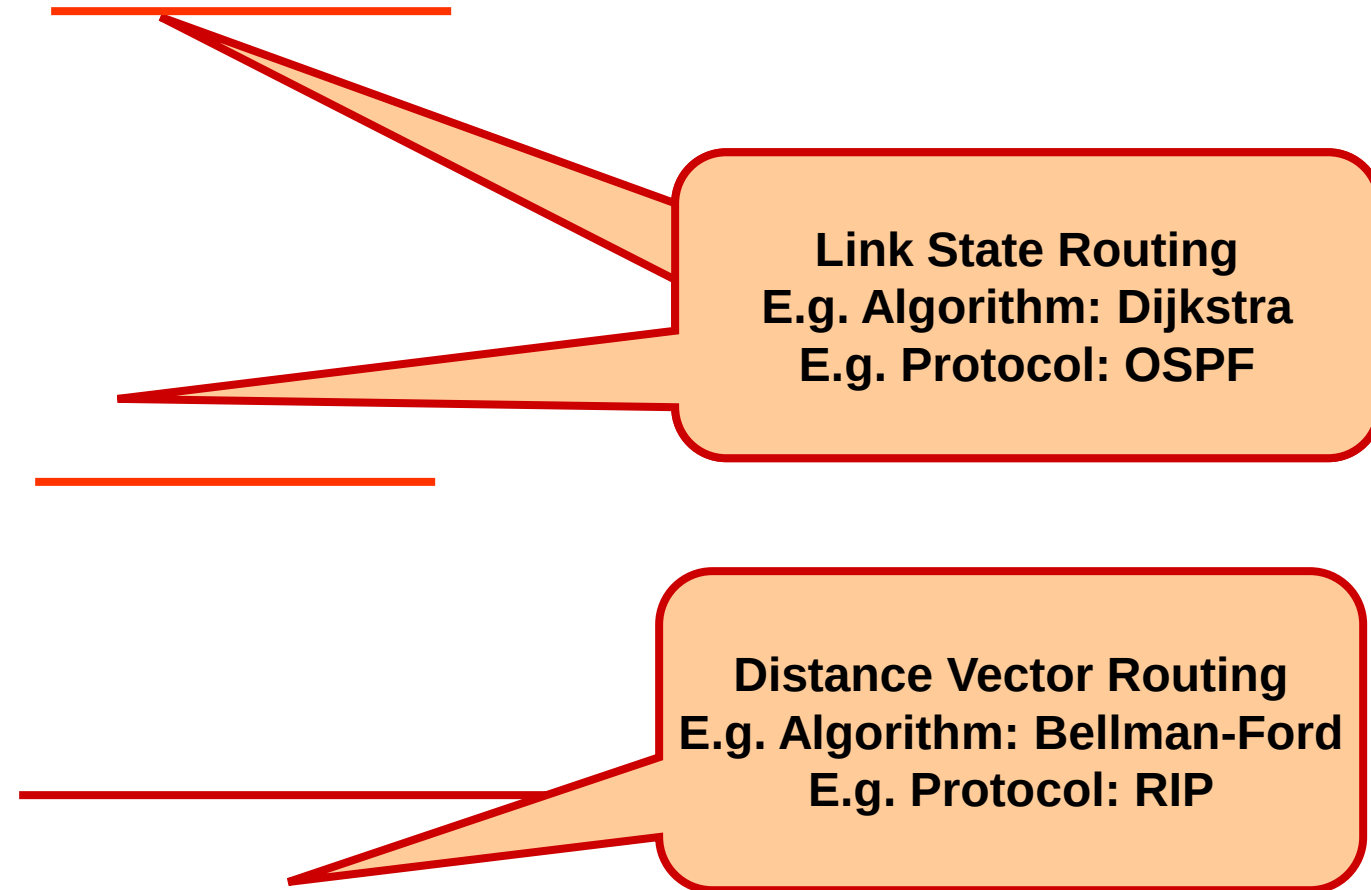


Distance Vector Practice problems

ANUSHA N

Routing Requires Knowing Network



Distance Vector Algorithm Overview

- Each router starts with a distance table consisting of the value “0” for itself and the value “infinity” for every other destination
- Each router will transmit its distance vector to each of its neighbors whenever the information changes (as well as when a link to a neighbor first comes up)
- Each router saves the most recently received distance vector from each of its neighbors, and calculate its own distance vector, based on minimizing the cost to each destination

Distance Vector Algorithm

Bellman-Ford Equation (dynamic programming)

Define

$d_x(y) :=$ cost of least-cost path from x to y

Then

$$d_x(y) = \min_v \{c(x,v) + d_v(y)\}$$

where min is taken over all neighbors v of x

Distance Vector Algorithm

iterative:

- continues until no nodes exchange info.
- *self-terminating*: no “signal” to stop

asynchronous:

- nodes need *not* exchange info/iterate in lock step!

distributed:

- each node communicates *only* with directly-attached neighbors

Distance Table data structure

- each node has its own
- row for each possible destination
- column for each directly-attached neighbor to node
- example: in node X, for dest. Y via neighbor Z:

$$\begin{aligned} D^X(Y,Z) &= \text{distance from } X \text{ to } Y, \text{ via } Z \text{ as next hop} \\ &= c(X,Z) + \min_w \{D^Z(Y,w)\} \end{aligned}$$

Distance Routing: Overview

Iterative, asynchronous:

each local iteration caused by:

- local link cost change
- message from neighbor: its least cost path change from neighbor

Distributed:

- each node notifies neighbors *only* when its least cost path to any destination changes
 - neighbors then notify their neighbors if necessary

Each node:

wait for (change in local link cost of msg from neighbor)

recompute distance table

if least cost path to any dest has changed, *notify* neighbors

Distance Vector Routing Algorithm

- Distance Vector Routing is a dynamic routing algorithm.
- It works in the following steps
- It follows Bellman Ford Algorithm

Distance Vector Routing Algorithm

- **Step-01:**

- Each router prepares its routing table.

By their local knowledge, each router knows about

- All the routers present in the network
- Distance to its neighboring routers

- **Step-02:**

- Each router exchanges its distance vector with its neighboring routers.

- Each router prepares a new routing table using the distance vectors it has obtained from its neighbors.

Figure 22.14 *Distance vector routing tables*

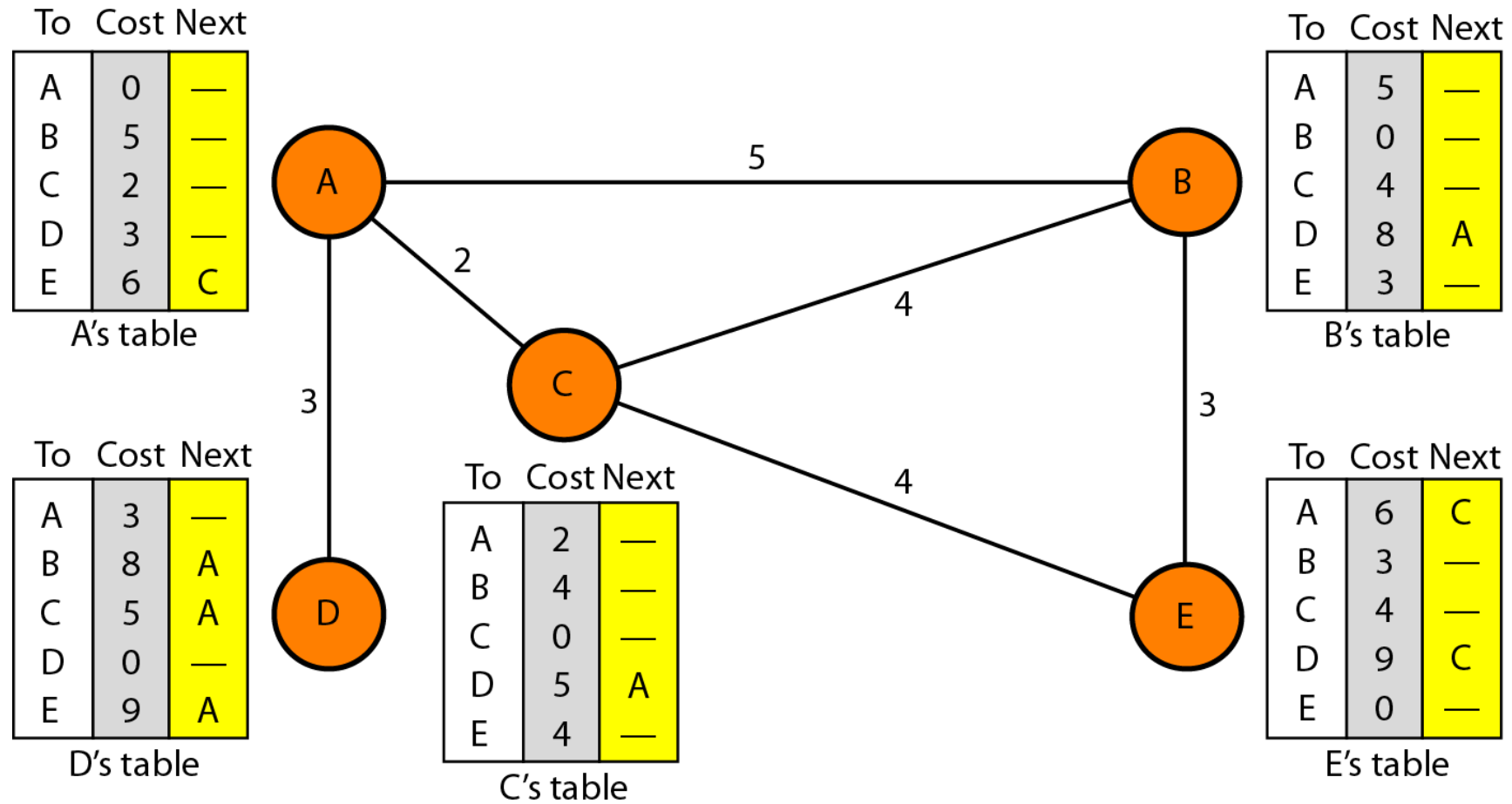
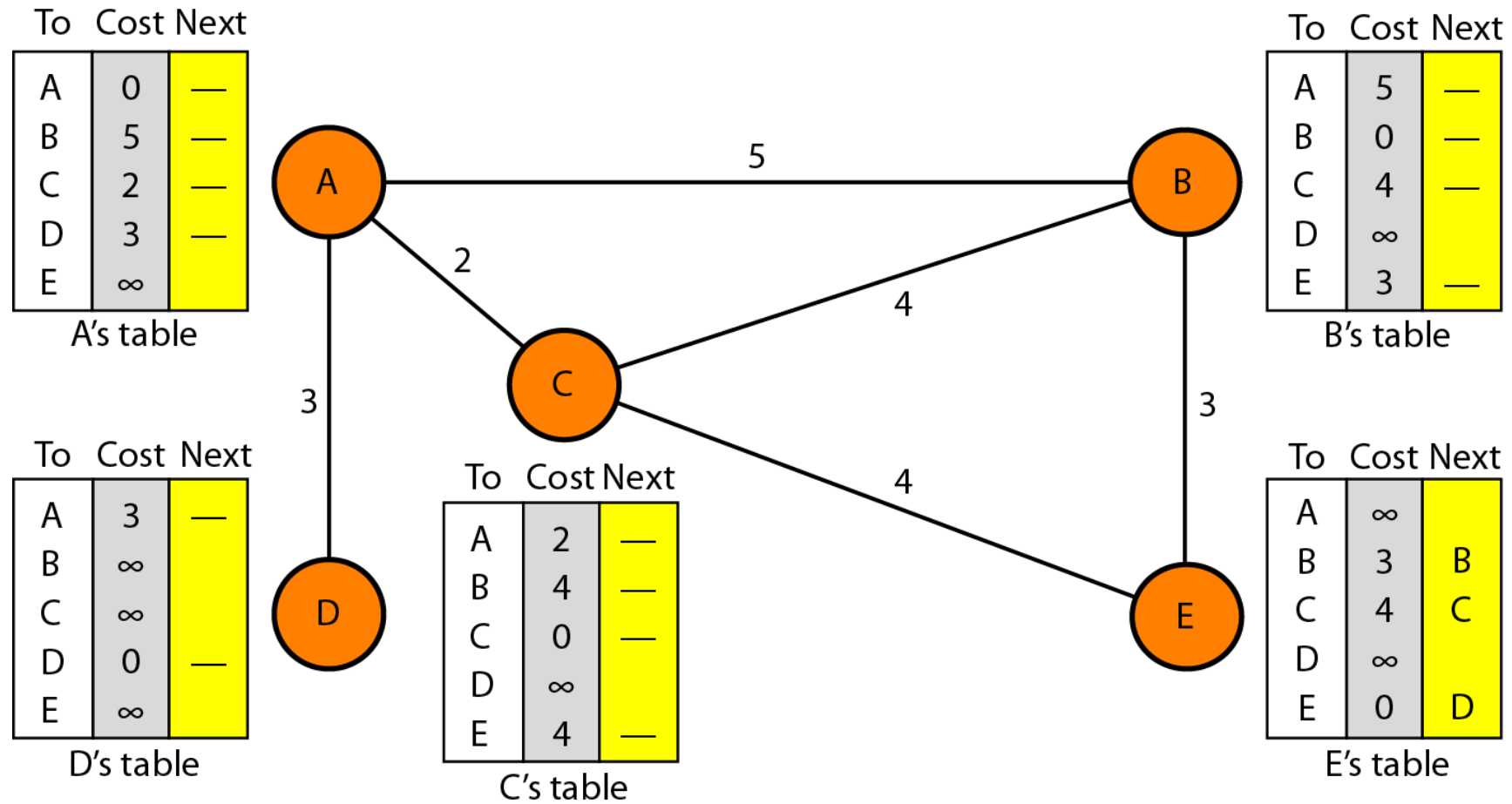


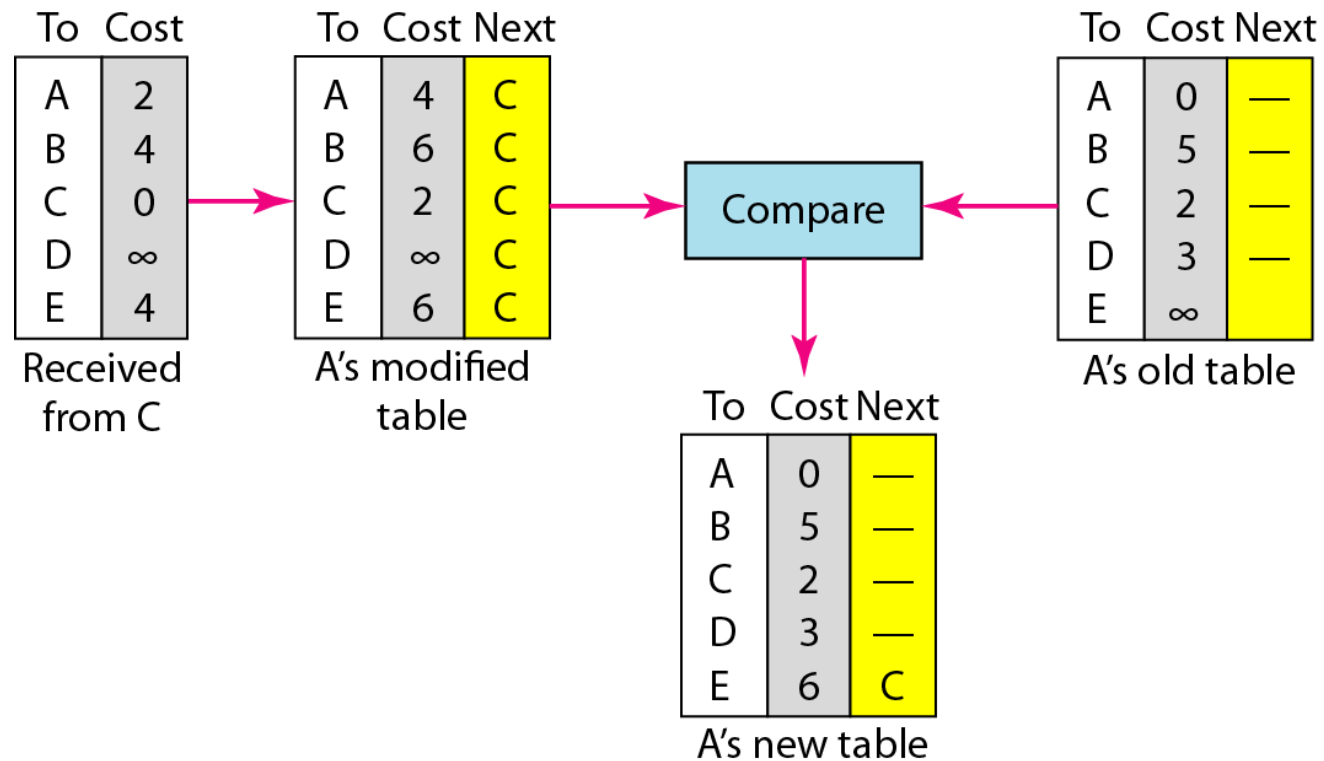
Figure 22.15 *Initialization of tables in distance vector routing*



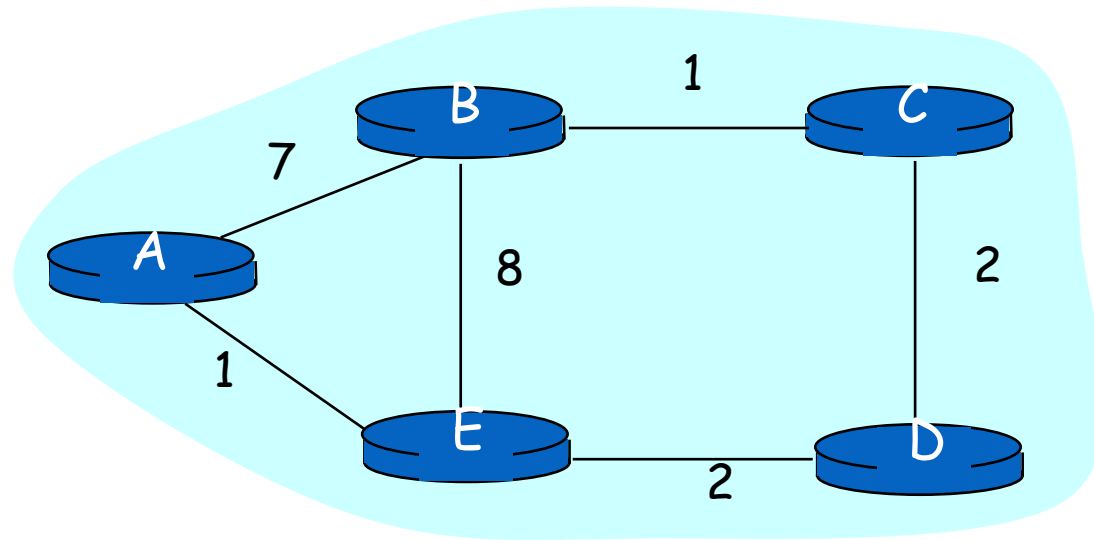
Note

In distance vector routing, each node shares its routing table with its immediate neighbors periodically and when there is a change.

Figure 22.16 *Updating in distance vector routing*

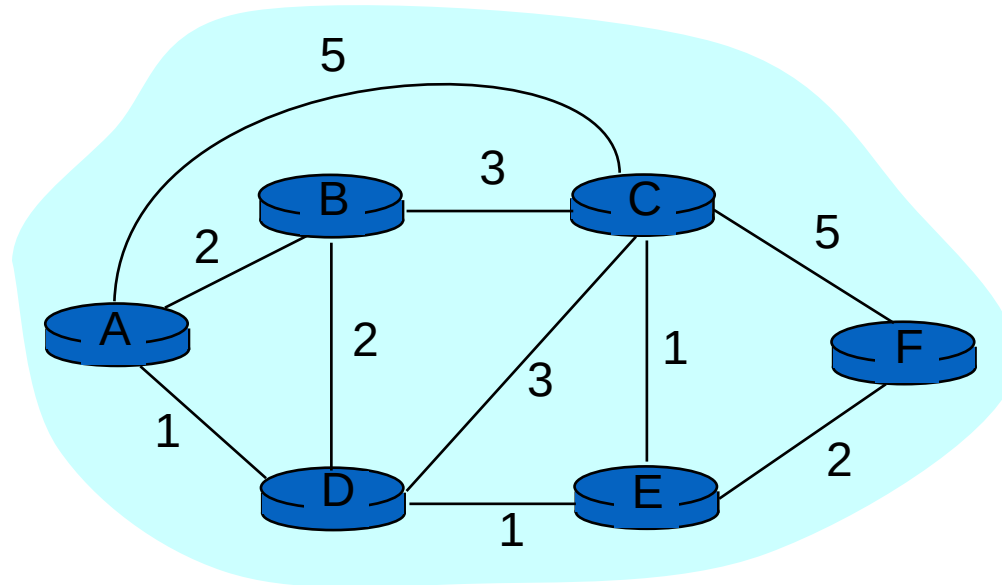


Distance Table: An Example

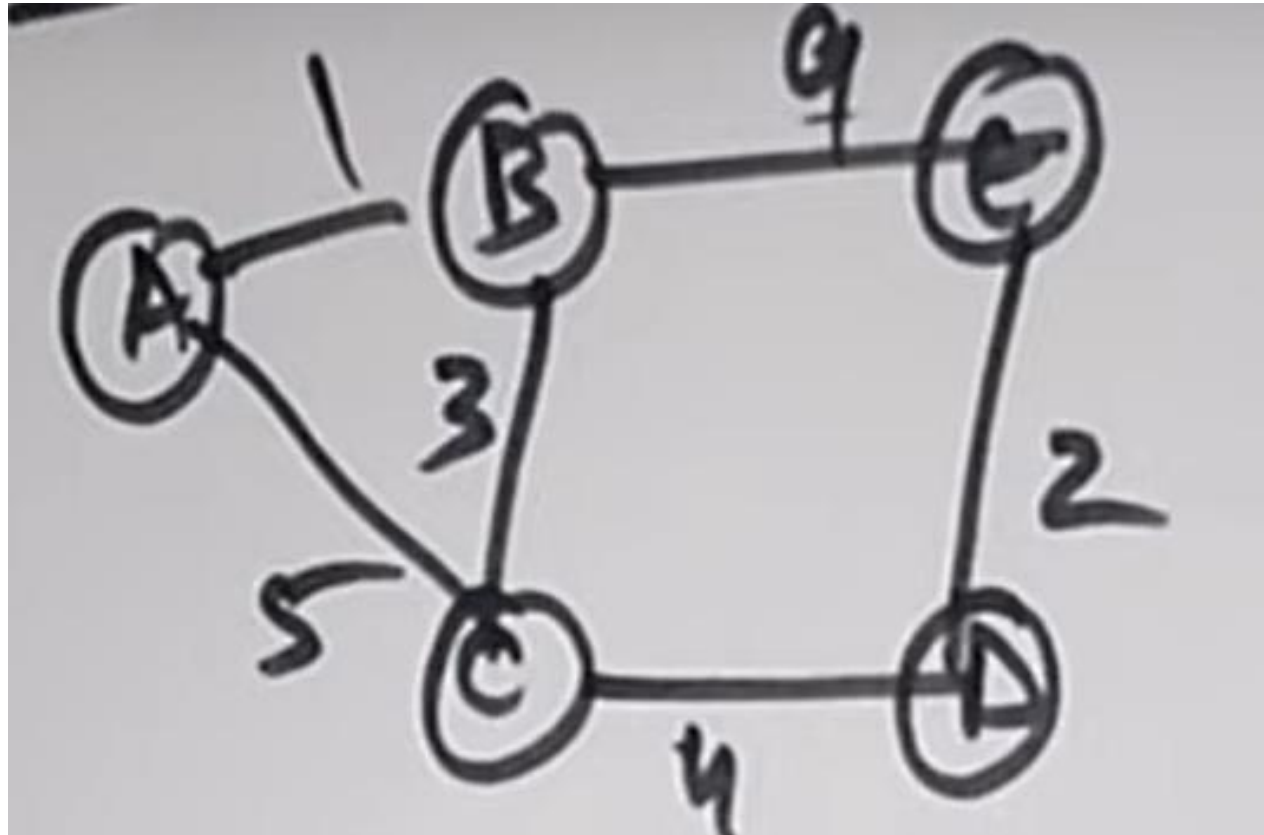


Modeling a Network

- Modeled as a *graph*
 - Routers $\Rightarrow$ nodes
 - Link $\Rightarrow$ edges
 - Possible edge costs
 - Hop
 - Delay
 - Congestion level
 -
- Goal of Routing
 - Determine “good” path from source to destination
 - “Good” usually means the lowest “cost” path
 - Where cost is usually hop-count or latency



Examples to solve



B to C:

$$B-C \Rightarrow 3 \checkmark$$

$$B-A-C \Rightarrow 6$$

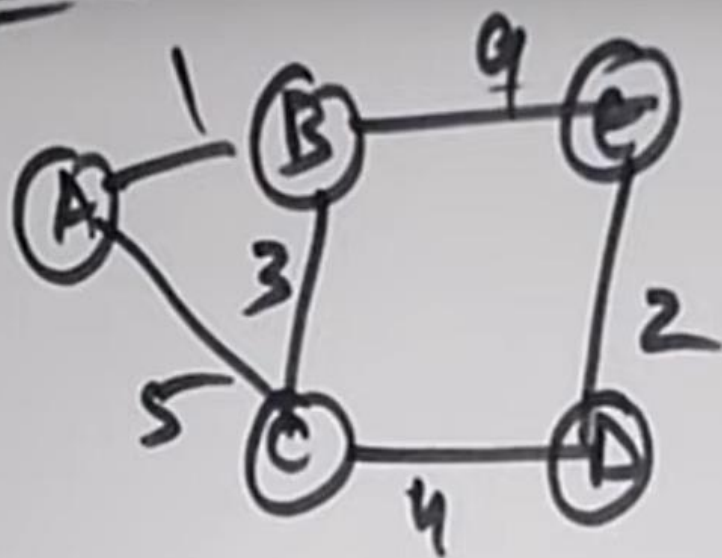
$$B-E-D-C \Rightarrow 15$$

B to D:

$$B-A-C-D \Rightarrow 1+5+4 \Rightarrow 10$$

$$B-C-D \Rightarrow 3+4=7 \checkmark$$

$$B-E-D \Rightarrow 9+2=11$$



Destination

A

C

D

Reference Node C

Example

D	C	H
A	5	A
B	3	B
D	4	D

Routing Table of C
(1)

To	A
A	0
B	1
C	5

Message from A
C to A: C = 5

D	C	H
A	5	A
B	3	B
D	4	D

Routing Table of C

D	C	H
A	5	A
B	3	B
D	4	D

Routing Table of C
(2)

To	B
A	1
B	0
C	3
E	9

Message from B
C to B: C = 3

D	C	H
A	4	B
B	3	B
D	4	D
E	12	B

Routing Table of C

D	C	H
A	4	B
B	3	B
D	4	D
E	12	B

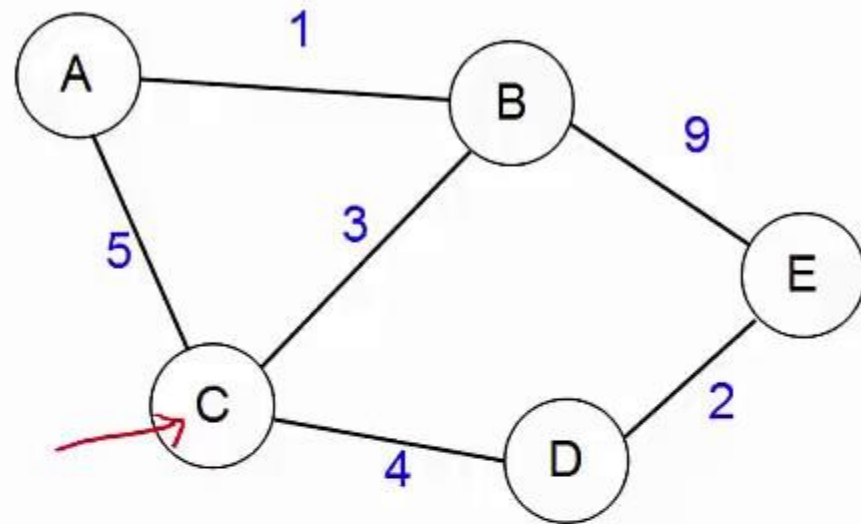
Routing Table of C
(3)

To	D
C	4
D	0
E	2

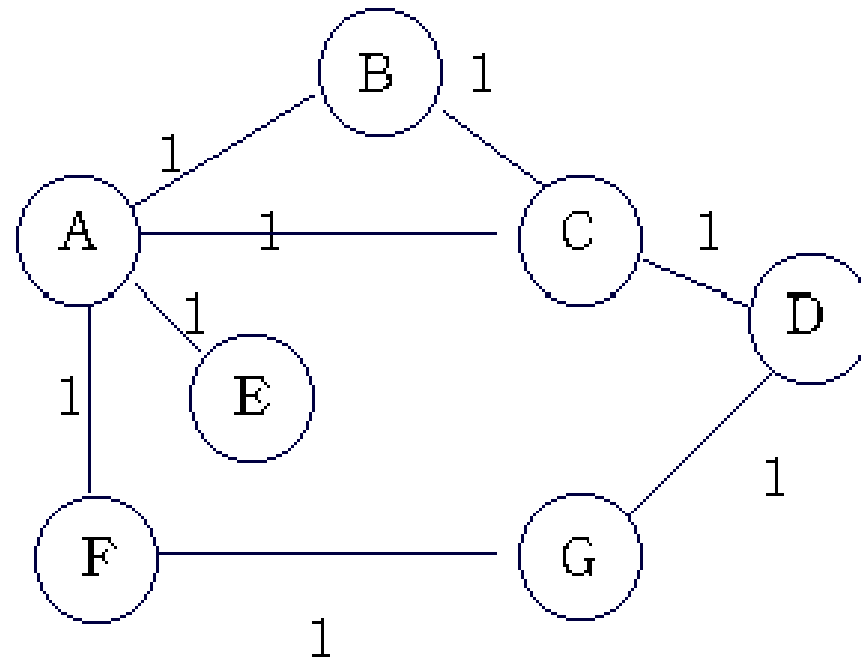
Message from D
C to D: C = 4

D	C	H
A	4	B
B	3	B
D	4	D
E	6	D

Routing Table of C



Examples to solve



Examples to solve

Information Stored at Node	Distance to Reach Node						
	A	B	C	D	E	F	G
A	0	1	1	?	1	1	?
B	1	0	1	?	?	?	?
C	1	1	0	1	?	?	?
D	?	?	1	0	?	?	1
E	1	?	?	?	0	?	?
F	1	?	?	?	?	0	1
G	?	?	?	1	?	1	0

Table 1. Initial distances stored at each node(global view).

Examples to solve

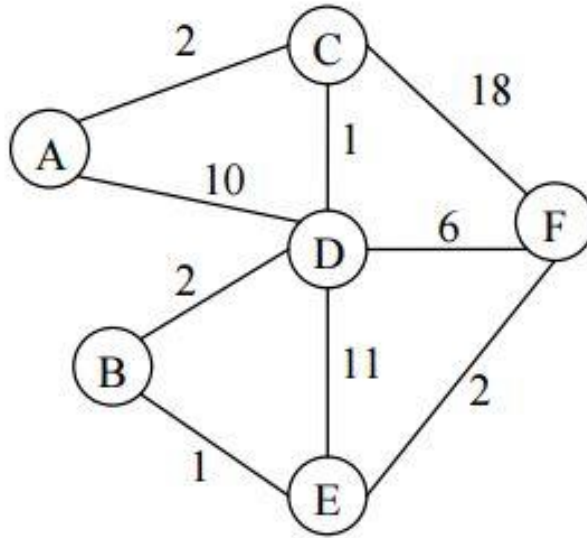
Information Stored at Node	Distance to Reach Node						
	A	B	C	D	E	F	G
A	0	1	1	2	1	1	2
B	1	0	1	2	2	2	3
C	1	1	0	1	2	2	2
D	2	2	1	0	3	2	1
E	1	2	2	3	0	2	3
F	1	2	2	2	2	0	1
G	2	3	2	1	3	1	0

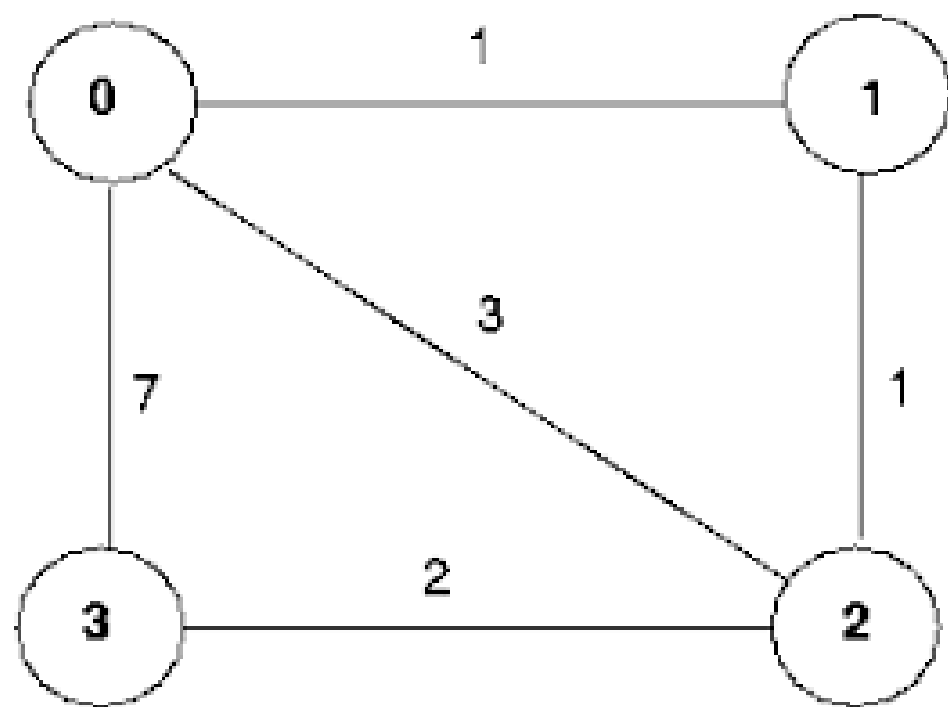
Examples to solve

Destination	Cost	NextHop
A	1	A
C	1	C
D	2	C
E	2	A
F	2	A
G	3	A

Table 3. Routing table maintained at node B.

Examples to solve





Examples to solve

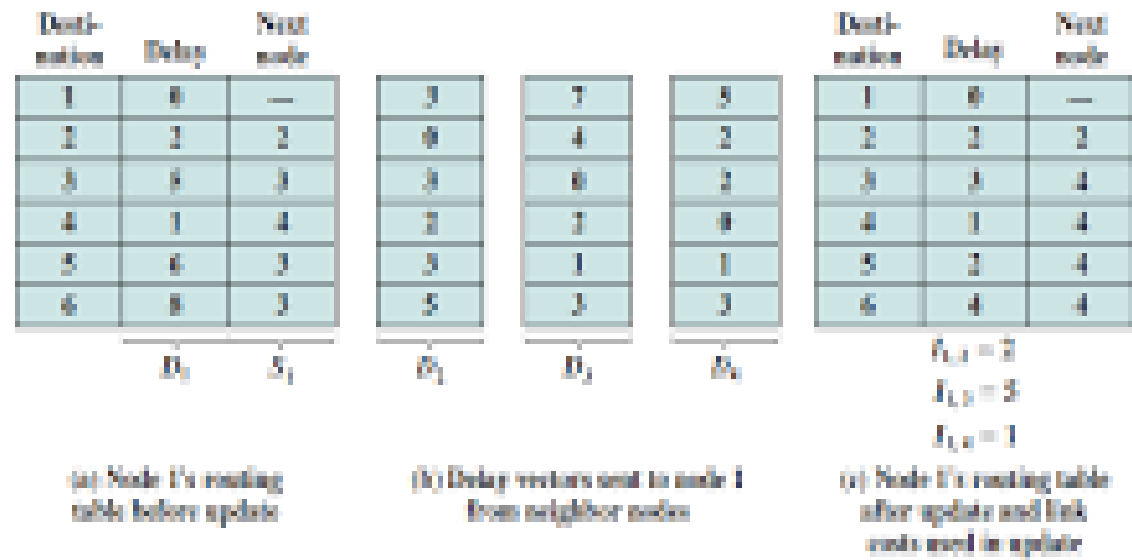


Figure 19.5 Original ARPANET Routing Algorithm

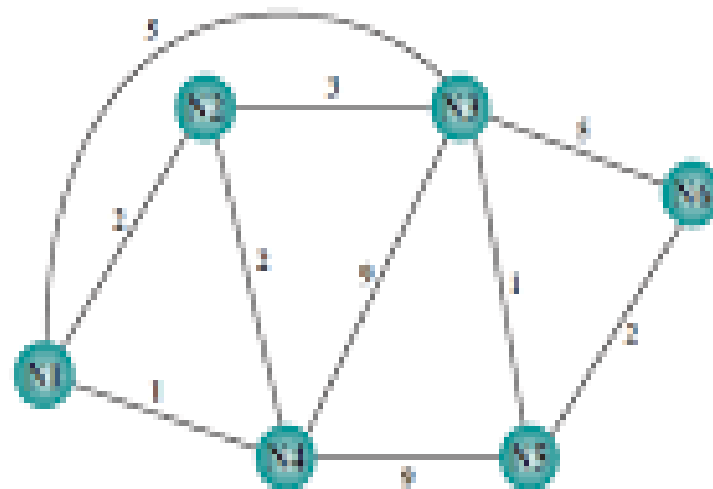
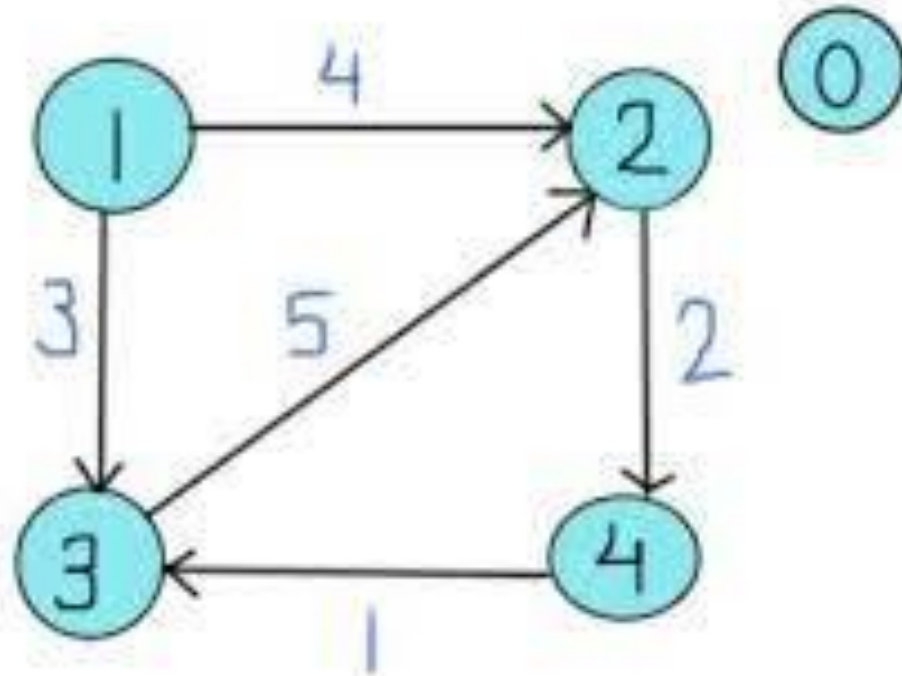
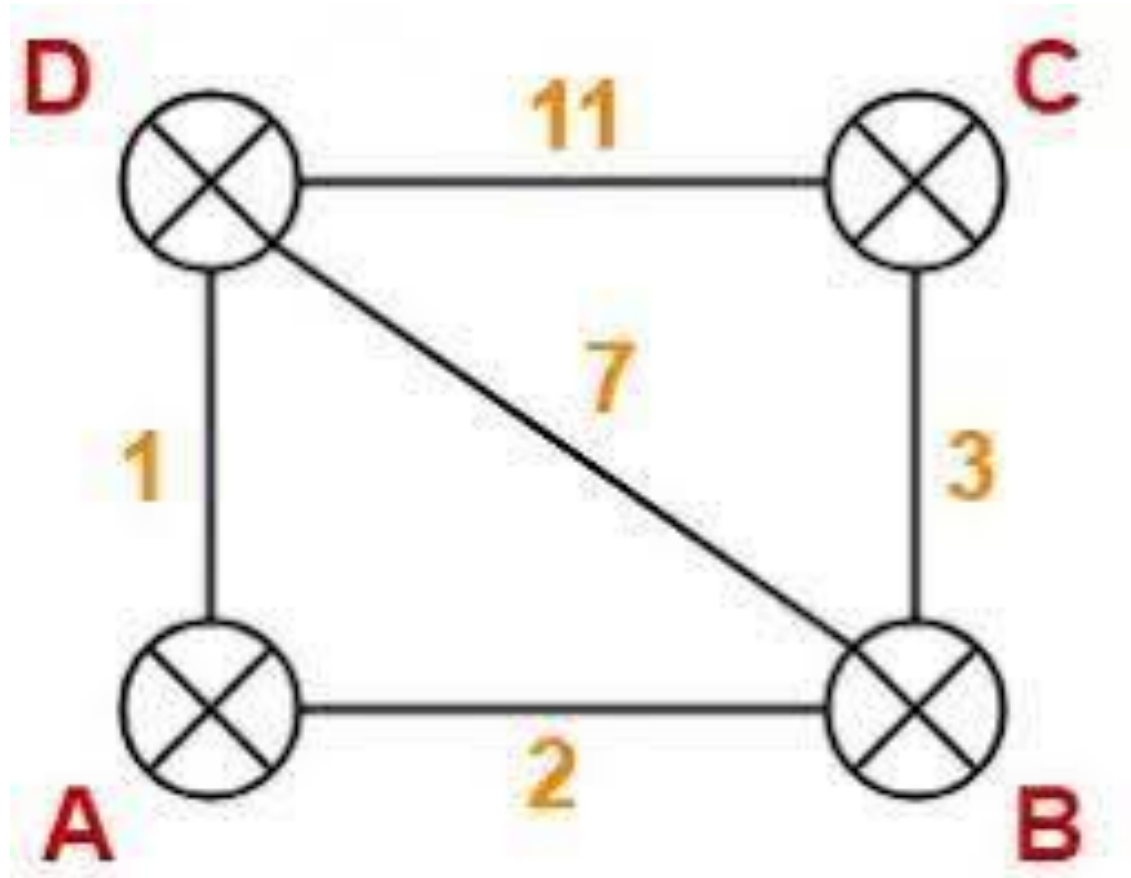


Figure 19.6 Network for Example of Figure 19.5a

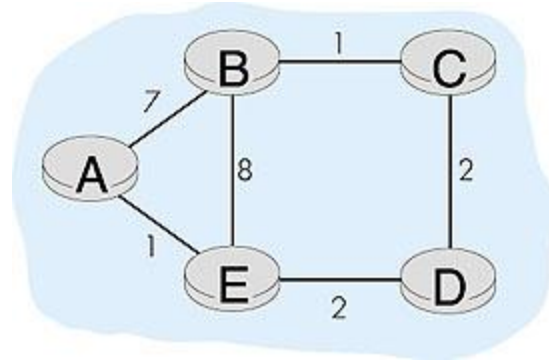
Examples to solve



Examples to solve



Examples to solve

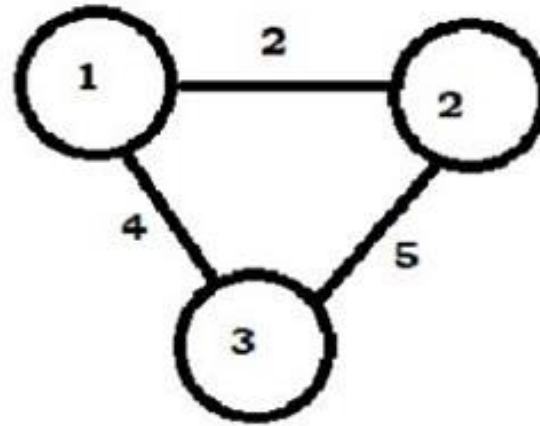


cost to destination via			
$D^2(i)$	A	B	D
A	①	14	5
B	7	8	⑤
C	6	9	④
D	4	11	②

Examples to solve

Node	Cost
1	0
2	2
3	4

Routing Table

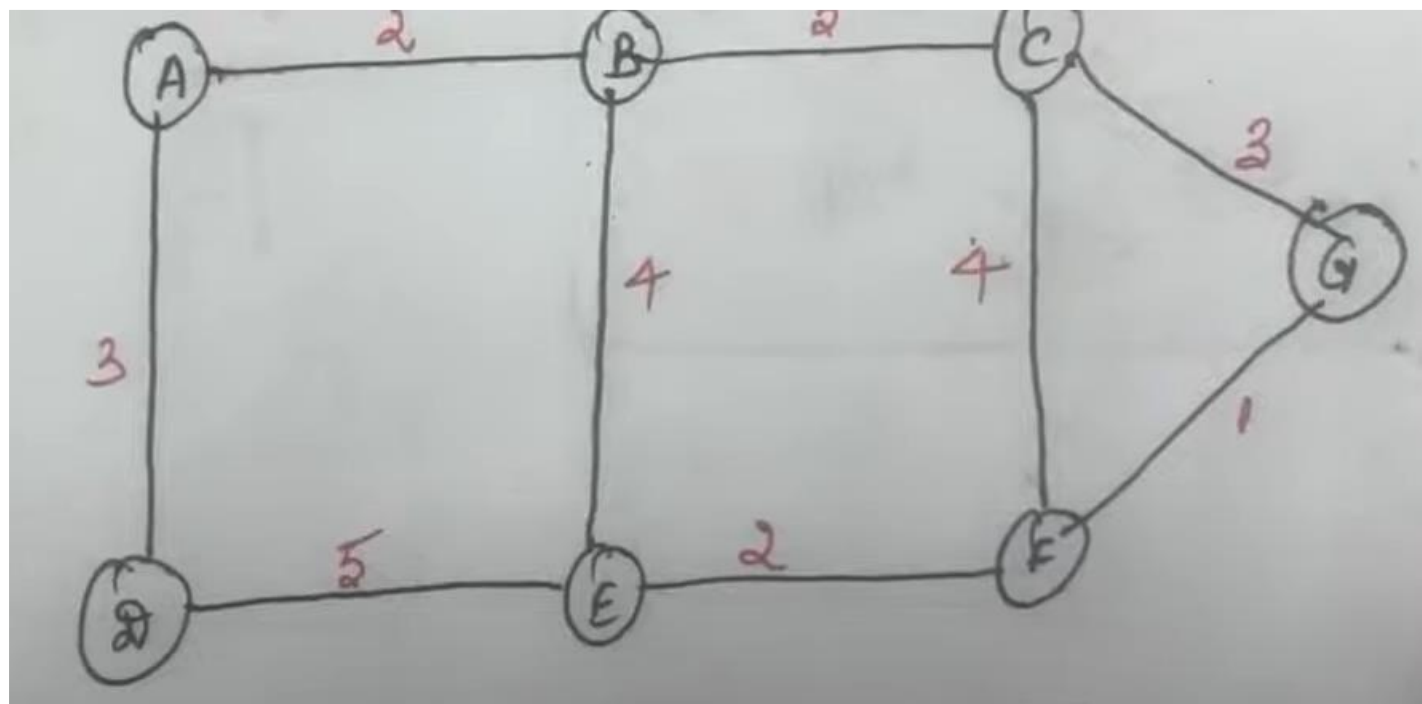


Node	Cost
1	2
2	0
3	5

Routing Table

Node	Cost
1	4
2	5
3	0

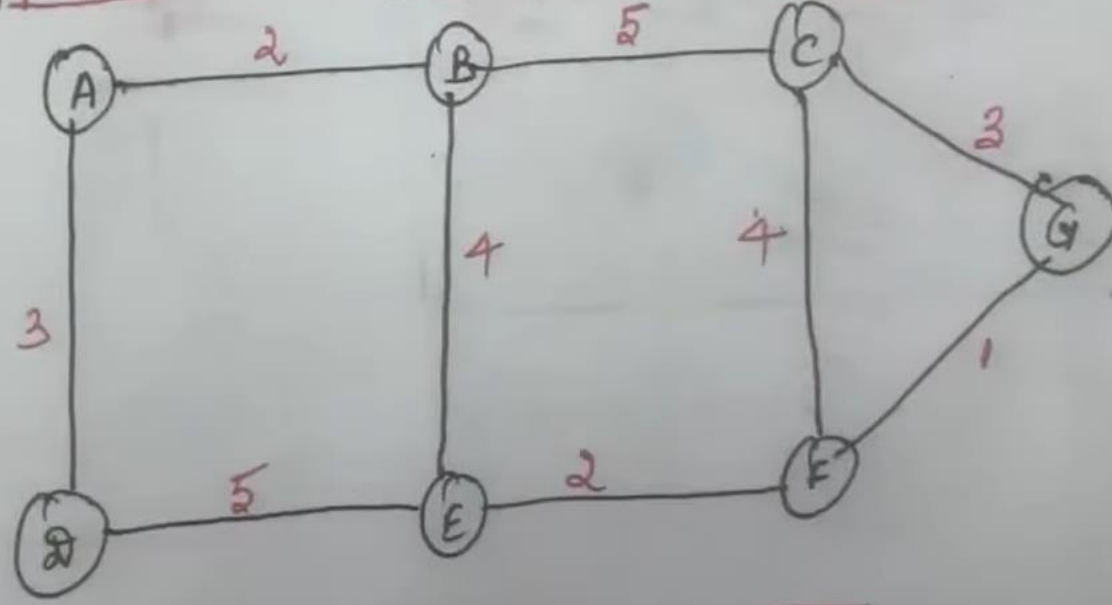
Routing Table



A	0
B	2
C	∞
D	3
E	∞
F	∞
G	∞

A	2
B	0
C	5
D	∞
E	4
F	∞
G	∞

A	∞
B	5
C	0
D	∞
E	∞
F	4
G	3



∞	A
∞	B
3	C
∞	D
∞	E
1	F
0	G

A	3
B	∞
C	∞
D	0
E	∞

A	∞
B	4
C	∞
D	5
E	0
F	2

A	∞
B	∞
C	4
D	∞
E	2
F	0

New B
2
0
5
5
4
∞
∞

=

old B
2
0
5
∞
4
∞
∞

A	
2	0
4	2
∞	∞
5	3
∞	∞
∞	∞
∞	∞

$$B[J] = \min(B[J], \underline{\underline{2 + A[J]}})$$

new B[J]

2
0
5
5
0
6
∞

old B

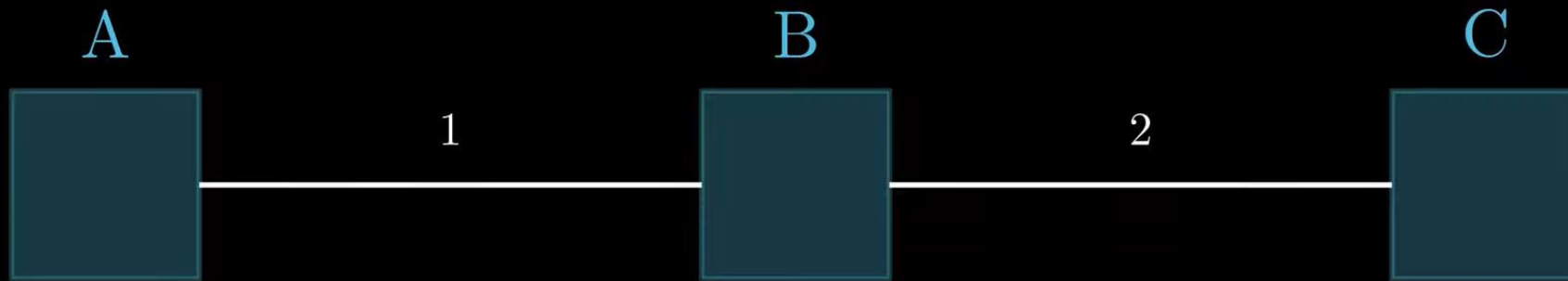
2 ✓
0 -
5 -
5
4
∞
∞

∞	∞
8	4
∞	∞
9	5
0	0
6	2
∞	∞

$$B[J] = \min(B[J], 4 + E[J])$$

B receives a copy of E's vector

Count to Infinity Problem

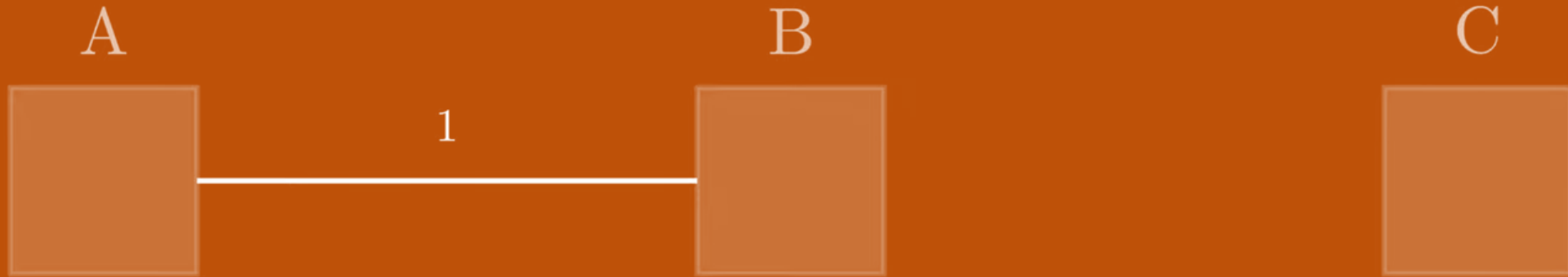


A		
To	Cost	Via
B	1	B
C	3	B

B		
To	Cost	Via
A	1	A
C	2	C

C		
To	Cost	Via
A	3	B
B	2	B

Count to Infinity Problem

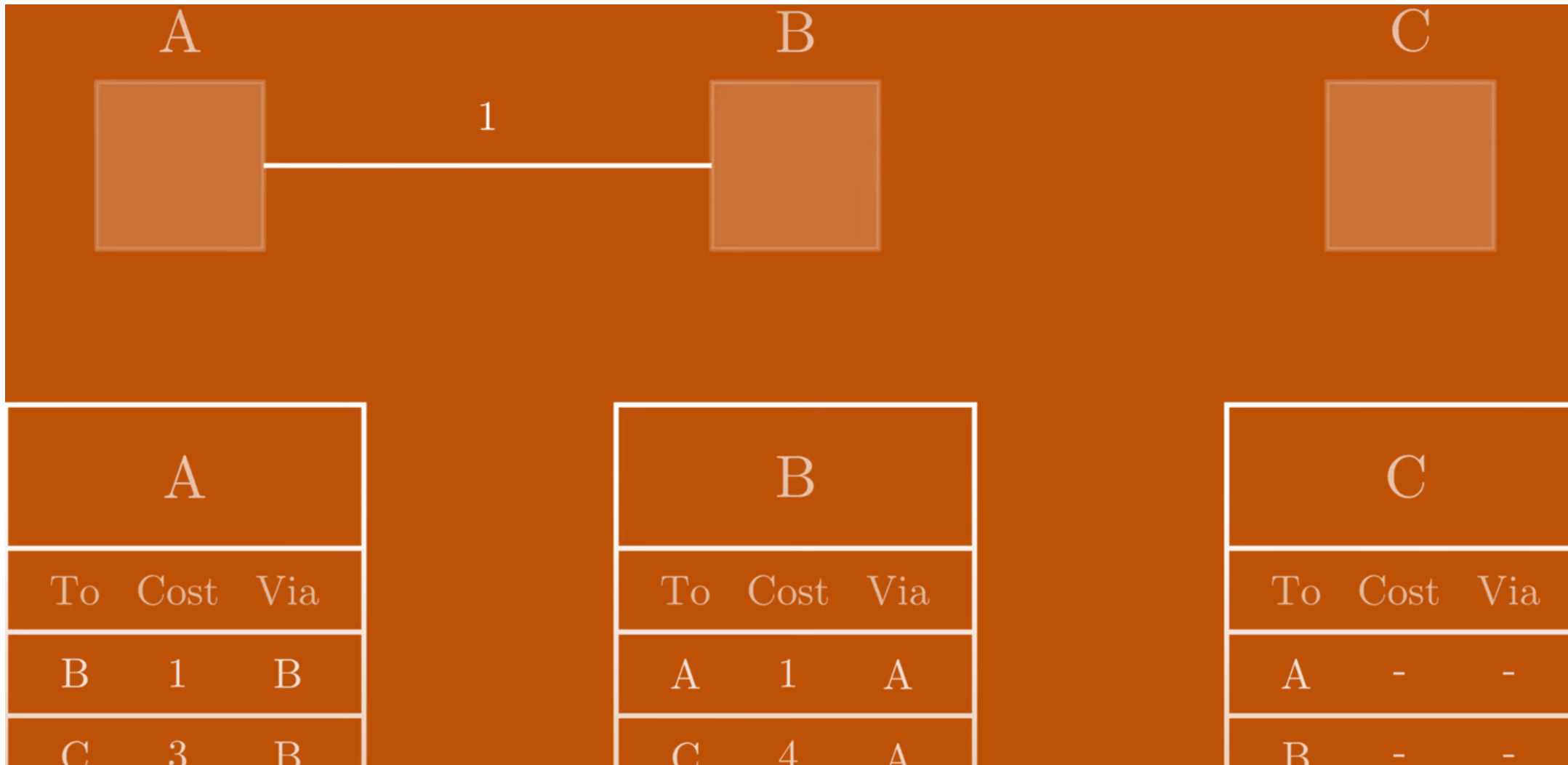


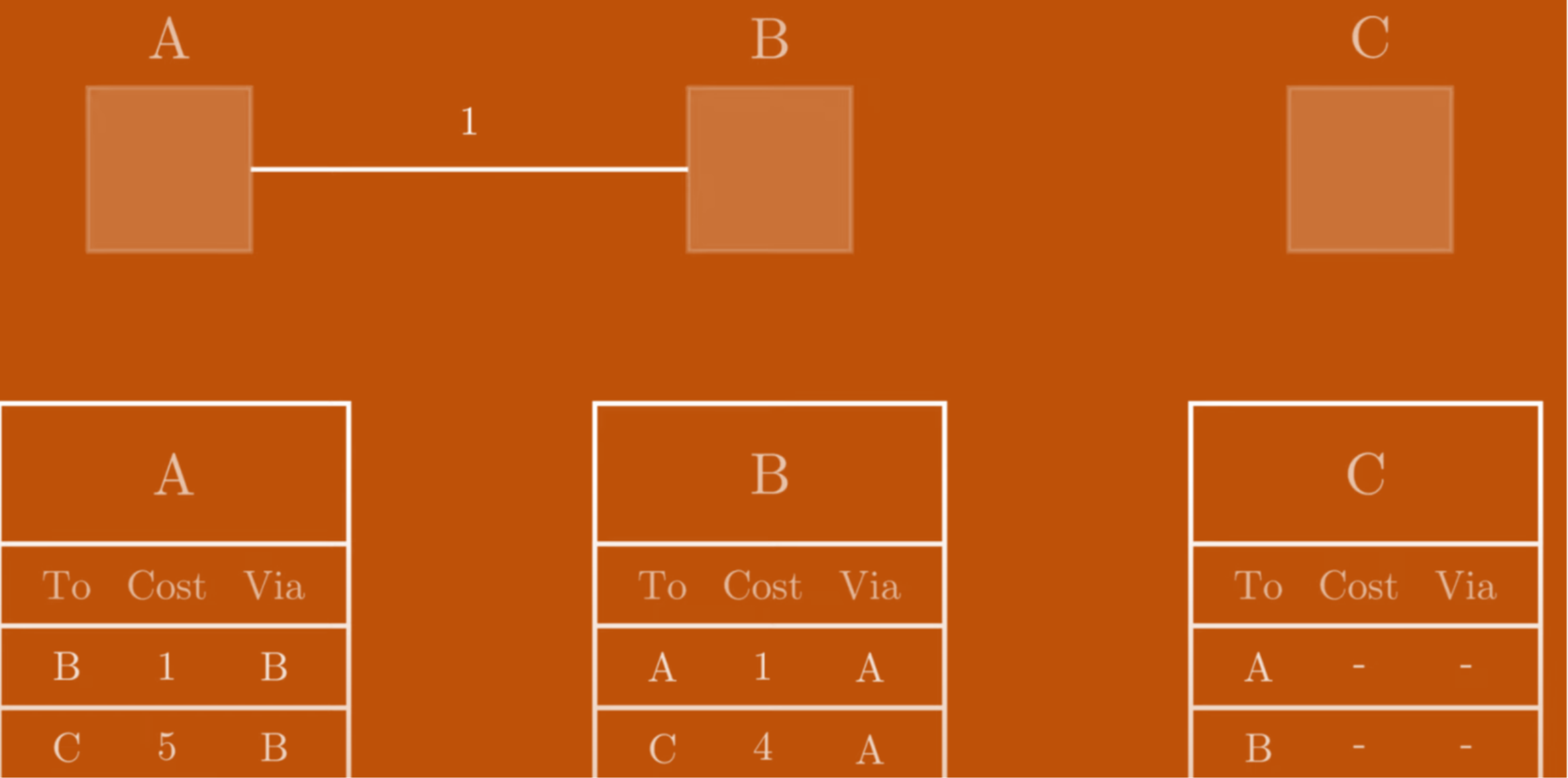
A		
To	Cost	Via
B	1	B
C	3	B

B		
To	Cost	Via
A	1	A
C	-	-

C		
To	Cost	Via
A	-	-
B	-	-

B thinks it can reach C via A and updates its table





Solution to Count to infinity

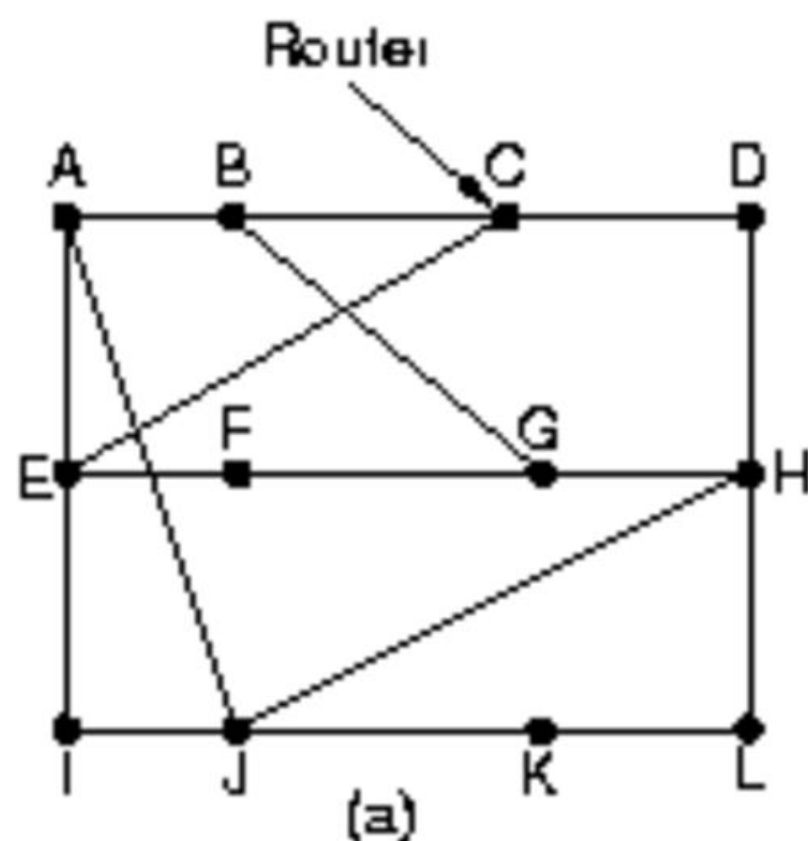
- Split Horizon
- Split Horizon and Poison Reverse

Solution to Count to infinity

- Split Horizon
- In this strategy, instead of flooding the table through each interface, each node sends only part of its table through each interface.
- If, according to its table, node B thinks that the optimum route to reach C is via A, it does not need to advertise this piece of information to A; the information has come from A (A already knows).
- Taking information from node A, modifying it, and sending it back to node A creates the confusion.
- In our scenario, node B eliminates the last line of its routing table before it sends it to A.
- In this case, node A keeps the value of infinity as the distance to C.
- Later when node A sends its routing table to B, node B also corrects its routing table.
- The system becomes stable after the first update: both node A and B know that C is not reachable.

Solution to Count to infinity

- Split Horizon and Poison Reverse
- Using the split horizon strategy has one drawback.
- Normally, the distance vector protocol uses a timer, and if there is no news about a route, the node deletes the route from its table.
- When node B in the previous scenario eliminates the route to C from its advertisement to A, node A cannot guess that this is due to the split horizon strategy (the source of information was A) or because B has not received any news about X recently.
- The split horizon strategy can be combined with the poison reverse strategy.
- Node B can still advertise the value for C, but if the source of information is A, it can replace the distance with infinity as a warning: "Do not use this value; what I know about this route comes from you."



(a) A subnet.

(b) Input from A, I, H, K ,
and the new
routing table for J .

					delay from J	
					↓	Line
To	A	I	H	K		
A	0	24	20	21	8	A
B	12	36	31	28	20	A
C	25	18	19	36	28	I
D	40	27	8	24	20	H
E	14	7	30	22	17	I
F	23	20	19	40	30	I
G	18	31	6	31	18	H
H	17	20	0	19	12	H
I	21	0	14	22	10	I
J	9	11	7	10	0	—
K	24	22	22	0	6	K
L	29	33	9	9	15	K

JA delay is 8	JI delay is 10	JH delay is 12	JK delay is 6
------------------------	-------------------------	-------------------------	------------------------

Vectors received from

New routing table for J	
----------------------------------	--

(b)